



TD 3: **Le polymorphisme** **en C++**



Exercice n°1

Soit le code C++ comportant les classes A et B suivantes :

```
class A
{
public: A(){cout<<"A::A()"<<endl;}
virtual ~A() { cout<< "A:: ~A()"<<endl;}
virtual void f1() {cout << "A::F1()" << endl;}
void f2(){cout << " A::f2()" << endl;}
};
```

```
class B : public A
{
public: B(){cout<<"B::B()"<<endl;}
~B(){cout<< "B:: ~B()"<<endl;}
void f1() {cout << " B::f1()" << endl;}
virtual void f2() {cout << " B::f2()" << endl;}
} ;
```

1. Quel est le résultat d’affichage du code suivant ? `int main(){ B obj; }`
2. Quelle est la différence entre une méthode virtuelle et une méthode non virtuelle au niveau de la convention d'appel ?
3. Quel est le résultat d’affichage du code suivant ?

```
int main()
{B *pB = new B;
A *pA = pB; /* A* pA;
pA=pB;
*/
pA->f1();
pB->f2();
pB->f1();
pA->f2();
}
```

Exercice n°2

- Une entreprise de Patagonie développe un moteur à huile de rutabaga, et dote plusieurs véhicules de ce moteur. Avant d'effectuer des tests grandeur nature, l'entreprise conçoit un programme de simulation du comportement des véhicules. Ce programme sera écrit en C++. On ne s'intéresse dans cet exercice qu'à une toute petite partie de ce programme. Le problème que l'on veut résoudre dans un premier temps est celui de la **représentation** des véhicules et du **calcul de la vitesse maximum** pouvant être atteinte par un véhicule. Tout 'véhicule' possède une **immatriculation (chaîne de caractères)** et un **poids à vide (réel)**. *Certains* véhicules peuvent transporter un chargement : on appelle alors **charge** le poids de ce chargement. La charge d'un véhicule ne doit pas dépasser un certain poids, que l'on appellera **charge maximale**, dépendant du *type* de véhicule. Les différents types de véhicules dotés du fameux moteur sont les suivants : les **petits bus**, les **camions citernes**, et les **camions bâchés**.
- la classe 'petit_bus' a un poids à vide de 4 tonnes, et peut atteindre une vitesse maximale de 150 km/h. Il ne possède pas de chargement (le poids des passagers est considéré comme négligeable par rapport au poids à vide).
- la classe 'camion_citerne' a un poids à vide de 3 tonnes et une charge maximale de 10 tonnes. Sa vitesse maximale dépend de sa charge :
 - 130 km/h si la charge est nulle
 - 110 km/h si la charge est inférieure ou égale à 1 tonne
 - 90 km/h si la charge est supérieure à 1 tonne et inférieure ou égale à 4 tonnes
 - 80 km/h pour une charge supérieure à 4 tonnes.
- la classe 'camion_bâché' a un poids à vide de 4 tonnes et une charge maximum de 20 tonnes. Sa vitesse maximale dépend également de sa charge (mais à charge égale, un camion-citerne a une vitesse maximale plus faible, car le liquide qu'il transporte est plus instable qu'un chargement solide):
 - 130 km/h si la charge est nulle
 - 110 km/h si la charge est inférieure ou égale à 3 tonnes
 - 90 km/h si la charge est supérieure à 3 tonnes et inférieure ou égale à 7 tonnes
 - 80 km/h au-delà.
- Les classes 'véhicule', 'petit_bus', 'camion_citerne', 'camion_bâché' possèdent une méthode affichage 'affichage' permettant d'afficher les caractéristiques de chaque véhicule. Les ingénieurs de l'entreprise fournissent une formule permettant de calculer la consommation de carburant d'un véhicule en fonction de sa vitesse et de son poids total (poids à vide + charge éventuelle) $\text{consommation} = (\text{vitesse-en-km/h})/10 + (\text{poids_total-en-tonnes})$. Cette formule est la même pour tous les types de véhicules, la méthode au calcul de consommation de carburant correspondante est : `float consommation (int v, int p)`, qui retourne la consommation (en litres pour 100 km) d'un véhicule de poids p allant à une vitesse v.

Travail à faire

Question1

Donnez en c++ l'interface et l'implémentation des classes 'Véhicule', 'petit_bus', 'camion_citerne', 'camion_bâché'.

Question2

On s'intéresse maintenant à la définition d'une classe **Convoi**, permettant de gérer un convoi de véhicules. Cette classe contient une collection de véhicules hétérogènes. L'ajout des véhicules se fait toujours à la fin. La classe **Convoi** doit savoir calculer la **vitesse maximale** d'un convoi, sachant que cette vitesse correspond à *la plus petite* des vitesses maximales des véhicules du convoi. Cette classe possède aussi les méthodes permettant l'ajout et la suppression de véhicules. Donnez l'interface et l'implémentation de la classe '**Convoi**' (pensez à utiliser le conteneur Vector de STL).

Question 3

Donnez la partie du code permettant de :

- créer un convoi 'C',
- créer deux camions bâchés
- créer un petit bus
- créer trois camions citerne
- ajouter ces véhicules à 'C'
- afficher les descriptions et la vitesse maximale de chaque véhicule de 'C'.

Exercice n° 3

On se propose d'implémenter, en C++, une partie d'un logiciel de gestion du courrier postal. Il s'agit d'implémenter l'ensemble des classes permettant cette gestion et d'ajouter toutes les méthodes nécessaires à leur bon fonctionnement. Il est à noter que tout au long de cet exercice les attributs des classes à définir doivent être non publics. Nous commençons par considérer les objets postaux. Ces objets sont de deux catégories : lettres et colis. Tous les objets postaux ont les caractéristiques suivantes :

- une origine,
- une destination,
- un code postal,
- un poids (en grammes),
- un volume (en m³),
- un taux de recommandation (égal à 0, 1 ou 2).

On doit pouvoir calculer leur tarif d'affranchissement, leur tarif de remboursement, et afficher une chaîne qui décrit l'objet.

Les lettres peuvent en outre avoir un certain caractère d'urgence.

Le tarif d'affranchissement d'une lettre se calcule de la manière suivante. Le tarif de base est de 0.5 UM (unité monétaire) auquel s'ajoutent cumulativement :

- 0.5 UM si le taux de recommandation est 1, 1.5 UM si le taux de recommandation est 2,
- 0.30 UM si c'est une lettre urgente. Le tarif de remboursement est de :
- 0 UM si le taux de recommandation est égal à 0,
- 1.5 UM si le taux est égal à 1,
- 15 UM si le taux est égal à 2.

La chaîne qui décrit une lettre a la forme suivante :

ouest/2/0.02/200

Lettre code postal/destination/taux de recommandation/caractère d'urgence

Par exemple : Lettre 7742/famille Kirik, igloo 5 banquise nord/1/ordinaire

Les colis possèdent les caractéristiques complémentaires suivantes :

- une déclaration de contenu (texte)

- une valeur déclarée (en UM)

Le tarif d'affranchissement d'un colis s'obtient par cumul des sommes suivantes :

- 2 UM dans tous les cas (tarif de base),

- 0.5 UM si le taux de recommandation, est 1, 1.5 UM si le taux de recommandation est 2 (comme pour les lettres),

- 3 UM de surtaxe si le colis dépasse 1/8 de m³. Le tarif de remboursement d'un colis est de :

- 10% de la valeur déclarée si le taux de recommandation est 1,

- 50% de la valeur déclarée si le taux de recommandation est 2,

- rien si le taux de recommandation est 0.

La chaîne qui décrit un colis a la forme suivante :

Colis code postal/destination/taux de recommandation/volume/ valeur déclarée Par exemple : Colis 7854/famille Kaya, igloo 10, terres

Travail à faire

Question 1

Donnez l'interface et l'implémentation des classes 'Objet_postal', 'Lettre' et 'Colis'.

Question 2

On s'intéresse maintenant à la définition d'une classe 'Sac_postal' permettant la description des sacs postaux. Un sac postal peut contenir un certain nombre d'objets postaux et dispose d'une capacité maximale. Cette capacité est de 0,5 m³ pour un sac ordinaire. On peut fabriquer des sacs d'une autre capacité, sur demande précisant la capacité voulue.

On doit pouvoir ajouter un objet dans un sac, s'il y rentre. On veut aussi connaître le volume occupé par un sac, et afficher son contenu. Cette classe possède aussi les méthodes permettant la suppression et la recherche d'un objet postal. Pour l'implémentation de la classe 'Sac_postal', on a besoin d'un conteneur permettant de stocker les objets postaux. A cet effet, on se propose initialement de définir et d'implémenter en c++, une classe représentant des tableaux dynamiques qu'on nomme 'Vecteur'. Cette classe 'Vecteur' peut gérer n'importe quel type d'éléments. Pour simplifier, on définira seulement les opérations sur les tableaux qui sont utiles pour le sac d'objet, c'est-à-dire ajouter un élément, supprimer un élément, savoir si un élément est présent ou non, savoir si le tableau est vide ou non et connaître la taille du tableau.

1. Donnez l'interface et l'implémentation de la classe 'Vecteur'.
2. Donnez l'interface et l'implémentation de la classe 'Sac_postal'(pensez à utiliser la classe 'Vecteur').

Question 3

Donnez la partie du code permettant de :

- créer un sac postal 'psac',
- créer deux lettres 'pl1' et 'pl2'.
- créer deux colis 'pc1' et 'pc2'.
- ajouter ces objets postaux à 'psac'.
- afficher le contenu de 'psac'.